2 Boolean Algebra

Introduction

- Boolean algebra is a system of mathematical logic.
- In our daily life, we come across so many two-valued problems such as 'true or false', 'good or bad', 'right or wrong', etc.
- All the above problems can be precisely solved by mathematical analysis of logic (logical algebra) and it is commonly known as Boolean algebra.
- Boolean algebra has many differences compared to ordinary algebra and binary number systems. Some of the key differences are as follows:

Grey Matter Alert!

Boolean algebra or logical algebra was invented by George Boole in 1854.

Boolean Algebra	Ordinary Algebra
A + A = A	A + A = 2A
A – A = A	A – A = A2

Table 2.1

Fun facts

1. In Boolean algebra 1 + 1 = 1, whereas in binary number system 1 + 1 = 0 with carry 1 and in ordinary algebra 1 + 1 = 2.
2. Any variable or function of variables in Boolean algebra can assume only two values, either a '0' or a '1', whereas the variables or functions in ordinary algebra can assume an infinite number of values.

Logic Gates

- The building blocks or building components of any digital circuit are called logic gates.
- Logic gates are used to perform or implement logical operations or functions in Boolean algebra.
- Some examples of logic gates are AND gate, OR gate, NOT gate, NAND gate, NOR gate, X-OR gate and X-NOR gate.

Rack Your Brain

Logic circuits which use three gates only, i.e., AND, OR and NOT are called AND/OR/INVERT (AOI logic).

Classification of logic gates:

The logic gates are classified into three categories, namely, the basic gates, the universal gates and the special purpose gates (derived gates).

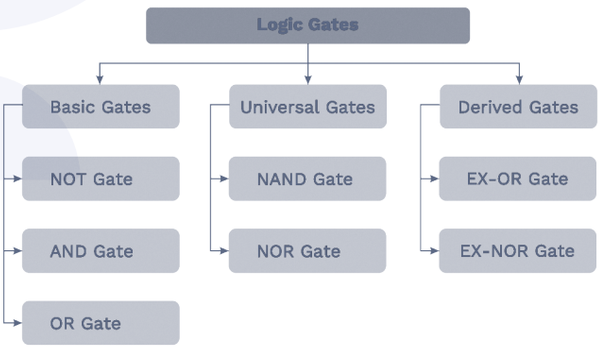


Fig. 2.1 Classification of Logic Gates

Rack Your Brain

AND gate is also called an all or nothing gate.

Applications of logic gates:

The major applications of logic gates are as follows:

- They are used to build complex devices such as binary counters, etc.

- In automatic control of machines, they are used for decision making.
- They are used in computers, calculators and digital measuring techniques.
- In musical instruments, games and domestic appliances, etc.

Basic Operators

The three basic operators to perform or implement a logical operation or function in Boolean algebra are:

1. AND operator
2. OR operator
3. NOT operator

1. **AND operator:** In Boolean algebra, the AND operator is defined by the use of multiplication (x) sign or dot (·) sign.
   The AND gate is called as AND operator, and it performs logical multiplication, i.e., AND operation.

Definitions

'A gate or a logic gate is a device that acts as a building block for digital circuits.'

For example:

If A · B = C, then

If A is true, (i.e., A = 1) and B is true, (i.e., B = 1), then only C will be true, (i.e., C = 1), else C will be false, (i.e., C = 0).

The different combinations of A and B are shown below:

Input		Output
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Table 2.2 AND operation



AND operator/AND gate

Fig. 2.2 AND Operation

2. **OR operator:** In Boolean algebra, a plus (+) sign is used for the OR operator.
   The OR gate is called as OR operator, and it performs logical addition.
   For example:
   If A + B = C, then
   If A is true, (i.e., A = 1) or if B is true, (i.e., B = 1), then C will also be true, (i.e., C = 1). C will be false, (i.e., C = 0), only when both A and B are false, (i.e., A and B = 0).

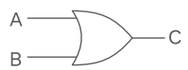
Rack Your Brain

'An OR gate is also called an any or all gate.'

The different combinations of A and B are shown below:

Input		Output
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Table 2.3 OR operation



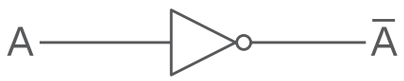
OR operator/OR gate

Fig. 2.3 OR Operation

3. **NOT operator:** In Boolean algebra, the not operator is defined by the use of an over bar (–) or apostrophe (').
   The NOT gate is called as NOT operator, and it performs the inversion operation.
   For example:
   If A = 0, (i.e., false), then $\overline{A} = 1$, (i.e., true).

Input	Output
A	$\overline{A}$
0	1
1	0

Table 2.4 NOT operation



NOT operator/NOT gate

Fig. 2.4 Truth Table of NOT Gate

Axioms and Laws of Boolean Algebra

- Axioms or postulates of Boolean algebra are a set of logical expressions that can be accepted without proof and upon which a set of useful theorems can be built.
- Actually, axioms are the definitions of the three basic logic operators and their operations that we have already discussed: AND, OR and NOT.

Laws of Boolean algebra:

1. Laws of complementation (NOT laws)
   Complement means to invert, i.e., to change 0s to 1s and vice-versa. The symbol is an over-bar.
   Five laws of complementation are as follows:
   Law 1: $\overline{0} = 1$
   Law 2: $\overline{1} = 0$
   Law 3: If $A = 0$, then $\overline{A} = 1$
   Law 4: If $A = 1$, then $\overline{A} = 0$
   Law 5: $\overline{\overline{A}} = A$
2. AND laws
   Four AND laws are as follows:
   Law 1: $A \cdot 0 = 0$
   Law 2: $A \cdot 1 = A$
   Law 3: $A \cdot A = A$
   Law 4: $A \cdot \overline{A} = 0$

Note:
The fourth law represents that if one input, (i.e., A) is logic 1, then the other input $\overline{A}$ would be logic 0, so the output is zero and vice-versa.

3. OR laws
   Four OR laws are as follows:
   Law 1: $A + 0 = A$
   Law 2: $A + 1 = 1$
   Law 3: $A + A = A$
   Law 4: $A + \overline{A} = 1$
4. Commutative laws
   There are two commutative laws that allow the change in position of variables in OR and AND operations. These are:
   Law 1: $A + B = B + A$ [The order in which variables are ORed is immaterial]
   Law 2: $A \cdot B = B \cdot A$ [The order in which variables are ANDed is immaterial]

Note:
This law can be extended to any number of variables. For example:
$A \cdot B \cdot C = B \cdot C \cdot A = C \cdot A \cdot B = B \cdot A \cdot C$

5. Associative laws
   There are two associative laws that allow the grouping of variables.
   Law 1: $(A + B) + C = A + (B + C)$
   This law states that the way in which variables are grouped and ORed is immaterial.
   Law 2: $(A \cdot B) \cdot C = A \cdot (B \cdot C)$
   This law states that the way in which variables are grouped and ANDed is immaterial.

Fun facts

These laws can be extended to a number of variables. For example:

- $A + (B + C + D) = (A + B + C) + D = (A + B) + (C + D)$
- $A \cdot (B \cdot C \cdot D) = (A \cdot B \cdot C) \cdot D = (A \cdot B) \cdot (C \cdot D)$

6. Distributive laws
   There are three distributive laws that allow factoring or multiplying out of expressions.
   Law 1: $A \cdot (B + C) = AB + AC$
   Law 2: $A + BC = (A + B)(A + C)$
   Let us prove this:
   RHS $= (A + B)(A + C)$
   $= AA + AC + BA + BC$
   $= A + AC + AB + BC \quad (\because \ AA = A)$
   $= A(1 + C + B) + BC$
   $= A \cdot 1 + BC \quad (\because \ 1 + C + B = 1)$
   $= A + BC = LHS$
   Law 3: $A + \overline{A}B = A + B$
   Let us prove this:

RHS = A + B
= 1 · (A + B)
= (A + $\bar{A}$) (A + B) ($\because$ A + $\bar{A}$ = 1)
= AA + AB + $\bar{A}$A + $\bar{A}$B
= A + AB + 0 + $\bar{A}$B ($\because$ A$\bar{A}$ = 0)
= A(1 + B) + $\bar{A}$B
= A (1) + $\bar{A}$B ($\because$ 1 + B = 1)
= A + $\bar{A}$B
= LHS

7. Absorption laws

There are three absorption laws:

Law 1: A + (A · B) = A

Let us prove this:

LHS = A + (A · B)
= A(1 + B)
= A(1) ($\because$ 1 + B = 1)
= A
= RHS

Law 2: A · (A + B) = A

Let us prove this:

LHS = A · (A + B)
= A · A + A · B
= A + A · B ($\because$ AA = A)
= A (1 + B)
= A · 1 ($\because$ 1 + B = 1)
= A
= RHS

Law 3: A · ($\bar{A}$ + B) = AB

Let us prove this:

LHS = A · ($\bar{A}$ + B)
= A$\bar{A}$ + AB
= 0 + AB
= AB
= RHS

A + 0 = A	$\bar{\bar{A}}$ = A
A + 1 = 1	A + B = B + A
A + A = A	A · B = B · A
$A+\bar{A}=1$	A + AB = A
A · 0 = 0	$A+\bar{A}B=A+B$
A · 1 = A	(A + B) + C = A + (B + C)
A · A = A	(A · B) · C = A · (B · C)
$A\cdot\bar{A}=0$	A · (B + C) = A · B + A · C
$\bar{0}=1$	A + B · C = (A + B)(A + C)
$\bar{1}=0$	$A\cdot(\bar{A}+B)=AB$

Boolean algebra basic rules

Table 2.5 General Boolean Algebra Expression

Demorganisation

The transformation $\overline{A+B}=\bar{A}\cdot\bar{B}$ and $\overline{A\cdot B}=\bar{A}+\bar{B}$ can be extended to complicated expressions by the following three steps:

Step 1: The entire function is initially complemented.

Step 2: All AND operations are replaced by OR operations and vice-versa.

Step 3: Finally, complement the individual variables. This procedure is commonly known as Demorganisation.

Example 1:

Prove that $(A+\bar{B}+AB)(A+\bar{B})(\bar{A}B)=0$

Solution:

$LHS=(A+\bar{B}+AB)(A+\bar{B})(\bar{A}B)$

But A + AB = A

$LHS=(A+\bar{B})(A+\bar{B})(\bar{A}B)$

$=(AA+A\bar{B}+A\bar{B}+\bar{B}\bar{B})(\bar{A}B)$

But A • A = A and $\bar{B}\cdot\bar{B}=\bar{B}$ and $A\bar{B}+A\bar{B}=\bar{B}(A+A)=A\bar{B}$

$\therefore LHS=(A+A\bar{B}+\bar{B})(\bar{A}B)$

$=[A(1+\bar{B})+\bar{B}](\bar{A}B)$

But $(1+\bar{B})=1$

$\therefore LHS=[(A\cdot1)+\bar{B}](\bar{A}B)$

$\therefore LHS=(A+\bar{B})(\bar{A}B)$ since A • 1 = A

$=A\bar{A}B+\bar{A}B\bar{B}$

But $A\bar{A}=0$ and $B\bar{B}=0$

$\therefore$ LHS = 0 + 0 = 0

Example 2:

Simplify the following expression:

$$Y = \left(\overline{\overline{AB} + \overline{A} + AB}\right)$$

Solution:

$$Y = \left(\overline{\overline{AB} + \overline{A} + AB}\right)$$

But $\overline{AB} = \overline{A} + \overline{B}$ ...DeMorgan's second theorem

$\therefore \quad Y = \left(\overline{\overline{A} + \overline{B} + \overline{A} + AB}\right)$

But $\overline{A} + \overline{A} = \overline{A}$

$\therefore \quad Y = \left(\overline{\overline{A} + \overline{B} + AB}\right)$

Now use DeMorgan's first theorem, which states that

$\overline{A + B + C} = \overline{A} \cdot \overline{B} \cdot \overline{C}$

$\therefore \quad Y = \overline{\overline{A}} \cdot \overline{\overline{B}} \cdot \overline{AB}$

But $\overline{\overline{A}} = A$ and $\overline{\overline{B}} = B$

$\therefore \quad Y = A \cdot B \cdot \overline{AB}$

But $\overline{AB} = \left(\overline{A} + \overline{B}\right)$... DeMorgan's second theorem

$\therefore \quad Y = A \cdot B\left(\overline{A} + \overline{B}\right) = A\overline{A}B + AB\overline{B}$

$A\overline{A} = 0$ and $B\overline{B} = 0$

$\therefore \quad Y = 0 \cdot B + A \cdot 0$

$= 0 + 0$ ...since $0 \bullet B = 0$ and $A \bullet 0 = 0$

$\therefore \quad Y = 0$

Example 3:

Minimise the given Boolean function with the help of Boolean algebra rules

$$f(A,B,C,D) = \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}CD + \overline{A}B\overline{C}D + \overline{A}BCD \cdot$$
$$+ A\overline{B}\,\overline{C}\,\overline{D} + A\overline{B}\,\overline{C}D$$
$$+A\overline{B}CD + ABC\overline{D} + AB\overline{C}D + AB\overline{C}\,\overline{D}$$

Solution:

$$f(A,B,C,D) = \overline{A}\,\overline{B}\,\overline{C}D + \overline{A}\,\overline{B}CD + \overline{A}B\overline{C}D + \overline{A}BCD \cdot$$
$$+ A\overline{B}\,\overline{C}\,\overline{D} + A\overline{B}\,\overline{C}D$$
$$+A\overline{B}CD + ABC\overline{D} + AB\overline{C}D + AB\overline{C}\,\overline{D}$$
$$= \overline{A}\,\overline{B}D\left(\overline{C} + C\right) + \overline{A}BD\left(\overline{C} + C\right) + A\overline{B}\overline{C}$$
$$\left(\overline{D} + D\right) + AC\overline{D}\left(\overline{B} + B\right) + ABC\left(\overline{D} + D\right)$$
$$= \overline{A}\,\overline{B}D + \overline{A}BD + A\overline{B}\overline{C} + AC\overline{D} + ABC$$
$$= \overline{A}D\left(\overline{B} + B\right) + A\overline{C}\left(\overline{B} + B\right) + AC\overline{D}$$
$$= \overline{A}D + A\overline{C} + AC\overline{D}$$
$$= \overline{A}D + A\left(\overline{C} + C\overline{D}\right)$$
$$= \overline{A}D + A\left[\overline{C}\left(1 + \overline{D}\right) + C\overline{D}\right]$$
$$= \overline{A}D + A\left[\overline{C} + \overline{C}\overline{D} + C\overline{D}\right]$$
$$= \overline{A}D + A\overline{C} + A\overline{D}$$
$$= A\overline{C} + \overline{A}D + A\overline{D}$$
$$= A\overline{C} + \left(A \oplus D\right)$$

Example 4:

Simplify the following function using Boolean algebra

$X = \overline{A}\,\overline{B}\overline{C} + A\overline{C}\overline{D} + A\overline{B} + ABC\overline{D} + \overline{A}\,\overline{B}C$

Solution:

$$X = \overline{A}\,\overline{B}\overline{C} + A\overline{C}\overline{D} + A\overline{B} + ABC\overline{D} + \overline{A}\,\overline{B}C$$
$$= \overline{A}\,\overline{B}\overline{C} + \overline{A}\,\overline{B}C + A\overline{C}\overline{D} + A\overline{B} + ABC\overline{D}$$
$$= \overline{A}\,\overline{B}\left(\overline{C} + C\right) + A\overline{C}\overline{D} + A\overline{B} + ABC\overline{D}$$

But $\overline{C} + C = 1$

$\therefore X = \overline{A}\,\overline{B} + A\overline{B} + A\overline{C}\overline{D} + ABC\overline{D}$

$$= \overline{B}\left(\overline{A} + A\right) + A\overline{C}\overline{D} + ABC\overline{D}$$

But $\overline{A} + A = 1$

$\therefore X = \overline{B} + A\overline{C}\overline{D} + ABC\overline{D}$

$$= \overline{B} + A\overline{C}\overline{D}\left(B + \overline{B}\right) + ABC\overline{D}$$
$$= \overline{B} + AB\overline{C}\overline{D} + A\overline{B}\overline{C}\overline{D} + ABC\overline{D}$$
$$= \overline{B} + AB\overline{C}\overline{D} + ABC\overline{D} + A\overline{B}\overline{C}\overline{D}$$
$$= \overline{B} + AB\overline{D}\left(\overline{C} + C\right) + A\overline{B}\overline{C}\overline{D} = \overline{B} + AB\overline{D} + A\overline{B}\overline{C}\overline{D}$$
$$= \overline{B}\left(1 + A\overline{C}\overline{D}\right) + AB\overline{D}$$

$\therefore X = \overline{B} + AB\overline{D}$

Truth Table

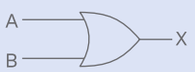
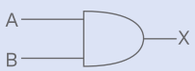
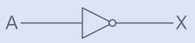
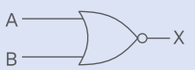
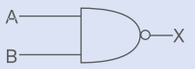
A logical table which shows the possible combinations of inputs and outputs in Boolean algebra is called as truth table.

Every logic gate, as well as its conversion, can be represented by a truth table. For example:

The truth table of an AND gate is expressed below.

Input		Output
A	**B**	**C**
0	0	0
0	1	0
1	0	0
1	1	1

Table 2.6 Truth Table for AND Operation

Logic Gate	Symbol	Boolean Function	Truth Table
OR		$X = A + B$	A B X 0 0 0 0 1 1 1 0 1 1 1 1
AND		$X = A \cdot B$	A B X 0 0 0 0 1 0 1 0 0 1 1 1
NOT OR INVERTER		$X = \overline{A}$	A X 0 1 1 0
NOR		$X = \overline{(A + B)}$	A B X 0 0 1 0 1 0 1 0 0 1 1 0
NAND		$X = \overline{(A \cdot B)}$	A B X 0 0 1 0 1 1 1 0 1 1 1 0
Exclusive OR or XOR		$X = A \oplus B$ or $X = \overline{A} \cdot B + A \cdot \overline{B}$	A B X 0 0 0 0 1 1 1 0 1 1 1 0

Logic Gate	Symbol	Boolean Function	Truth Table
Exclusive NOR or XNOR		$X = \overline{A \oplus B}$ or $X = A \cdot B + \overline{A}\,\overline{B}$	A B X 0 0 1 0 1 0 1 0 0 1 1 1

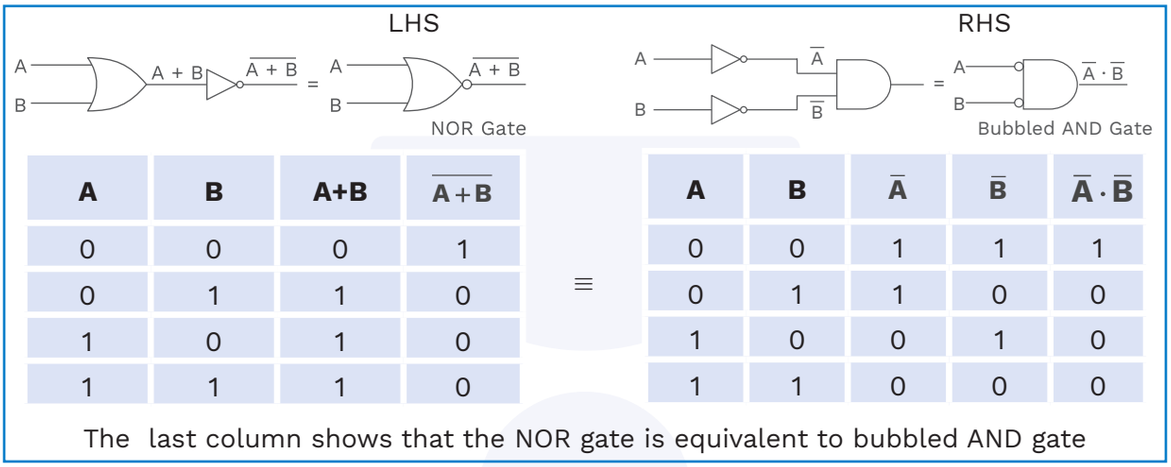
Table 2.7

Demorgan's Theorem

DeMorgan's theorem represents two most powerful laws in Boolean algebra.

Law 1: $\overline{A + B} = \overline{A} \cdot \overline{B}$

This law states that 'the complement of a sum of variables is equal to the product of their individual complements.' Schematically, each side of this law can be represented as:



A	B	A+B	$\overline{A+B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

≡

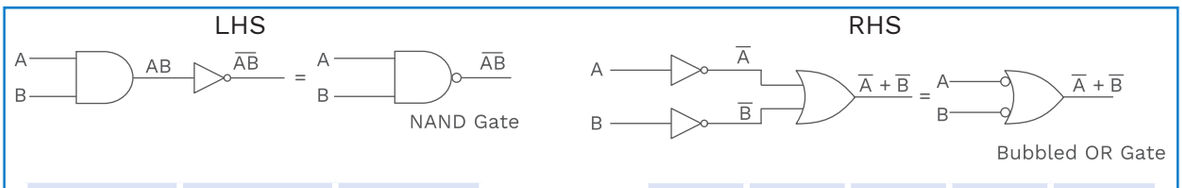
A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

The last column shows that the NOR gate is equivalent to bubbled AND gate

Fig. 2.5 Table Representing Demorgan's Theorem

Law 2: $\overline{A\,B} = \overline{A} + \overline{B}$

This law states that 'the complement of the product of variables is equal to the sum of their individual complements.' Schematically, each side of this law can be represented as:



A	B	A B
0	0	1
0	1	1
1	0	1
1	1	0

≡

A	B	$\overline{A}$	$\overline{B}$	$\overline{A} + \overline{B}$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	1
1	1	0	0	0

The last column shows that the NAND gate is equivalent to a bubbled OR gate

Fig. 2.6 Truth Table for NAND Operation

Example 5:

Demorganize $Y = \overline{(A + \overline{B})(C + \overline{D})}$.

Solution:

The given function is $Y = \overline{(A + \overline{B})(C + \overline{D})}$

On complementing the entire function, we get $(A + \overline{B})(C + \overline{D})$

Changing ANDs to ORs and ORs to ANDs, we get $A \cdot \overline{B} + C \cdot \overline{D}$

Now, on complementing the variables, we get $Y = \overline{A} \cdot B + \overline{C} \cdot D$

Example 6:

Demorganize $\overline{\overline{AB}(CD + \overline{E}F)(\overline{AB} + \overline{CD})}$.

Solution:

The given expression is:

Let $Y = \overline{\overline{AB}(CD + \overline{E}F)(\overline{AB} + \overline{CD})}$

$Y = \overline{\overline{AB}} + \overline{CD + \overline{E}F} + \overline{\overline{AB} + \overline{CD}}$

$Y = AB + (\overline{CD} \cdot \overline{\overline{E}F}) + (\overline{\overline{AB}} \cdot \overline{\overline{CD}})$

$Y = AB + (\overline{C} + \overline{D}) \cdot (E + \overline{F}) + ABCD$

SOP, POS and Canonical Forms of Boolean Function Representation

SOP (sum of products):

It is a summation of product terms.

For example:

$Y = AB + \overline{A}BC + \overline{A}\overline{B}C$

POS (product of sums):

It is a product of sum terms.

For example:

$Y = (A + B) \cdot (A + C) \cdot (\overline{A} + B)$

Canonical forms:

Standard SOP	Standard POS
In a SOP expression, if all the product terms contain all the variables of the function, then it is called as standard SOP.	In a POS expression, if all the sum terms contain all the variables of the function, then it is called as standard POS.
For example: $F(A, B, C) = \overline{A}BC + AB\overline{C} + A\overline{B}C + \overline{A}\overline{B}\overline{C}$ It is standard SOP because all the product terms contain all the three variables of the function.	For example: $F(A, B, C) = (A + \overline{B} + C) \cdot (\overline{A} + \overline{B} + C) \cdot (\overline{A} + B + \overline{C})$ It is a standard POS because all the sum terms contain all the three variables of the function.

Table 2.8

Minterms and Maxterms

- Each variable in a Boolean expression is a literal.
- Boolean variables can appear either in normal form (x) or complemented form ($\overline{x}$).
- A minterm is an AND combination of terms.
- A maxterm is an OR combination of terms.

Minterm: Each individual product term in a standard SOP is called as minterm.

For example:

Two variable functions f(A, B), the minterms for all the possible combinations are as follows:

Rack Your Brain

'A NAND gate is equivalent to a bubbled OR gate.'

Variables		Minterms
A	B	SOP
0	0	$\overline{A}\overline{B} \rightarrow m_0$
0	1	$\overline{A}B \rightarrow m_1$
1	0	$A\overline{B} \rightarrow m_2$
1	1	$AB \rightarrow m_3$

Table 2.9

Maxterm: Each individual sum term in a standard POS is called as maxterm.

For example:

Two variable functions f(A, B), the maxterms for possible combinations are as follows:

Variables		Maxterms
A	B	POS
0	0	$A + B \rightarrow M_0$
0	1	$A + \overline{B} \rightarrow M_1$
1	0	$\overline{A} + B \rightarrow M_2$
1	1	$\overline{A} + \overline{B} \rightarrow M_3$

Table 2.10

Example 7:

Represent the given Boolean function in minterms

$f(A,B) = AB + \overline{A}B$

Solution:

AB is represented as m3, and $\overline{A}B$ is represented as m1.

Previous Years' Questions

The output of a logic gate is '1' when all its inputs are at logic '0'. The gate is:

(A) a NAND or an Ex-OR gate

(B) a NOT or an EX-NOR gate

(C) a NOR or an EX-NOR gate

(D) an AND or an EX-OR gate

Solution: (C)

Hence, the given function can be represented as:

f(A, B) = ∑m(1, 3)

Representing functions with minterms:

Minterm number is the same as the row position in the truth table (starting with 0 at the top).

Let there be a truth table of a circuit as shown below, and we have to represent it by a Boolean function (in minterms).

A	B	C	f	
0	0	0	1	← m_0
0	0	1	1	← m_1
0	1	0	1	← m_2
0	1	1	0	
1	0	0	1	← m_4
1	0	1	1	← m_5
1	1	0	0	
1	1	1	0	

Table 2.11 Table Representing Min Terms

Rack Your Brain

'A NOR gate is equivalent to a bubbled AND gate.'

It can be represented by a Boolean function as:

$f = \overline{A}\overline{B}\overline{C} + \overline{A}\overline{B}C + \overline{A}B\overline{C} + A\overline{B}\overline{C} + A\overline{B}C$

f = m0 + m1 + m2 + m4 + m5

f = ∑m(0, 1, 2, 4, 5)

Representing functions with maxterms:

Maxterm number is the same as the row position in the truth table (starting with 0 at the top).

Let there be a truth table of a circuit as shown below, and we have to represent it by a Boolean function (in Maxterm).

A	B	C	f	
0	0	0	1	
0	0	1	1	
0	1	0	1	
0	1	1	⓪	← m_3
1	0	0	1	
1	0	1	1	
1	1	0	⓪	← m_6
1	1	1	⓪	← m_7

Table 2.12 Table Representing Maxterms

It can be represented by a Boolean function as:

$$f = (A + \overline{B} + \overline{C}) \cdot (\overline{A} + \overline{B} + C) \cdot (\overline{A} + \overline{B} + \overline{C})$$

f = M3 · M6 · M7

f = πM(3, 6, 7)

Conversion between canonical forms:

- It is quite easy to convert minterms into maxterms and vice versa.
- Firstly, we will represent it by a Boolean function (in minterms) and convert all AND operations with OR operations and vice versa.
- Finally, complement the individual variables.
- Let us understand the conversion with the help of an example; a truth table of a digital circuit is shown below.

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Table 2.13

$$f = \overline{A}BC + AB\overline{C} + ABC$$

↓

f = m3 + m6 + m7 = ∑(3, 6, 7)

↓ Conversion in

maxterms

f = M0 M1 M2M4 M5 = πM(0, 1, 2, 4, 5)

↓

$$f = (A + B + C) \cdot (A + B + \overline{C}) \cdot (A + \overline{B} + C)$$
$$\cdot (\overline{A} + B + C) \cdot (\overline{A} + B + \overline{C})$$

Nand and Nor Implementation

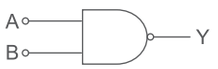
NAND and NOR gates are called as universal gates. They can be used to produce any other logic or Boolean function with the NAND and NOR gates being minimal.

NAND and NOR gates are called as minimal sets in themselves since they can be used individually or together to construct many other logic circuits.

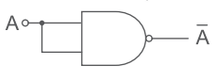
Hence, the complete set of operations of the main logic gates can be defined as:

- AND, OR and NOT (a full set)
- AND and NOT (a complete set)
- OR and NOT (a complete set)
- NAND (a minimal set)
- NOR (a minimal set)

Implementation of logic functions using only NAND gates:

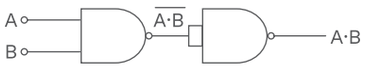


NAND Gate symbol

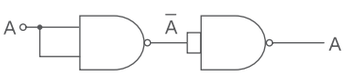


NOT Gate/Inverter

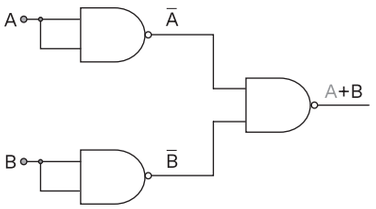
Fig. 2.7 Representation of NOT Gate



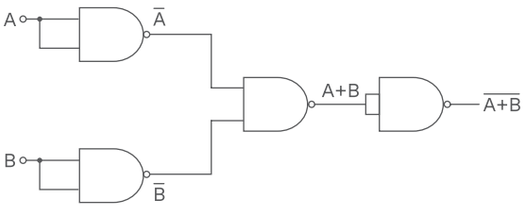
AND Gate



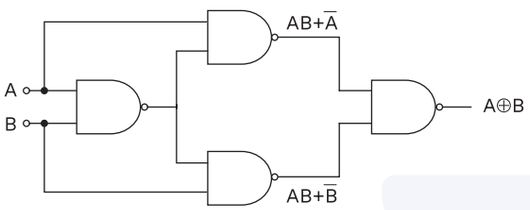
Buffer



OR Gate

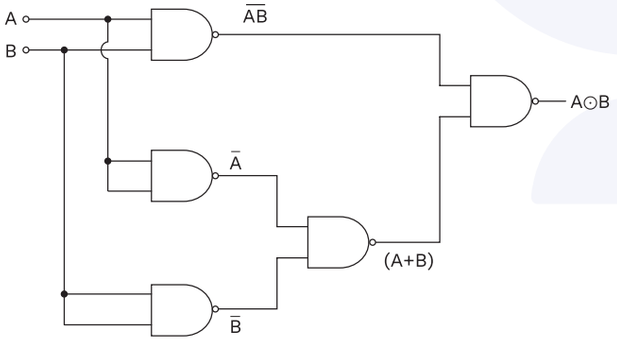


NOR Gate



Exclusive-OR Gate

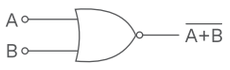
Fig. 2.8 Design Of Standard Logic Gates Universal Gates



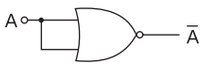
Exclusive-NOR Gate

Fig. 2.9 Design Of EX-NOR Gate Using Universal Gate

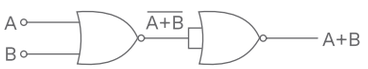
Implementation of logic functions using only NOR gates:



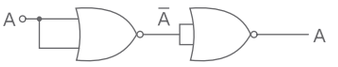
NOR Gate Symbol



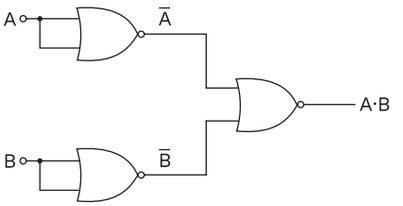
NOT Gate/Inverter



OR Gate

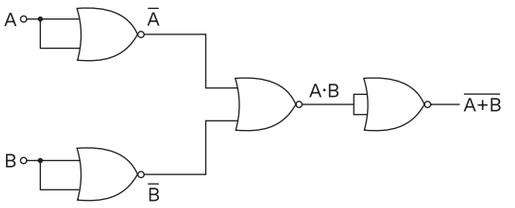


Buffer

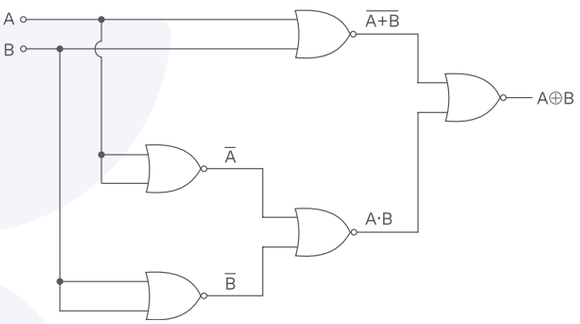


AND Gate

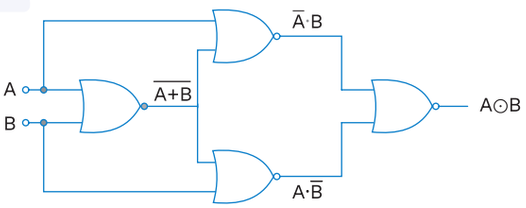
Fig. 2.10 Design of AND Gate Using NOR Gate



NAND Gate



Exclusive-OR Gate



Exclusive-NOR Gate

Fig. 2.11 Design of EX-NOR Gate Using NOR Gate

Two level Nand-Nand Implementation

The simplified Boolean function of SOP expression can be implemented by using AND-OR gates. The AND-OR implementation is a two-level implementation. For such implementation, it is necessary to convert the logic diagram from AND-OR logic to NAND-NAND logic. In the figure below, two equivalent symbols for NAND gates are shown.

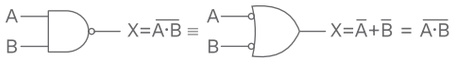


Fig. 2.12 Two Graphic Symbols for Nand Gates

NAND-NAND implementation:

The following steps are followed to convert AND-OR logic to NAND-NAND logic.

1. Simplify the given logical expression and, after that, convert it into the corresponding SOP form
2. Draw the AND-OR realisation.
3. Replace AND gate by NAND gate and OR gate by invert OR gate.
4. Replace invert OR gate by NAND gate.
5. Finally, draw the circuit using only NAND gates.
6. A bubble denotes complementation, and two bubbles along the same line represent double complementation. So, they should be removed.

Example 8:

Implement the given Boolean function with NAND-NAND gate logic.

$$F = AB + CD$$

Solution:

Step 1:

The function is implemented with AND-OR gates.

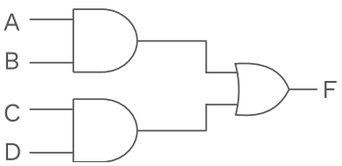


Fig. 2.13

Step 2:

The AND gates are replaced by NAND gates and OR gate is replaced by a NAND gate with an OR invert graphic symbol.

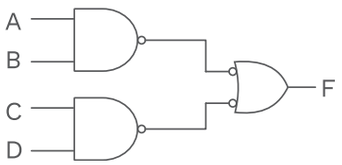


Fig. 2.14

Previous Years' Questions

Minimum number of 2 input NAND gates required to implement the function given below is:

$F = (\overline{X} + \overline{Y})(Z + W)$

(A) 4
(B) 5
(C) 6
(D) 7

Solution: (A)

Step 3:

The output NAND gate is redrawn with the AND invert graphic symbol.

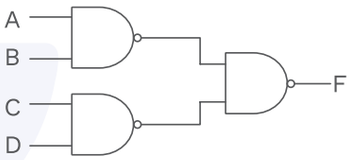


Fig. 2.15

The above figure can be verified algebraically. The function it implements can be easily converted to SOP using DeMorgan's theorem.

$$F = \overline{\overline{(A \cdot B)}\,\overline{(C \cdot D)}} = AB + CD$$

Two level NOR-NOR implementation

The two graphic symbols for NOR gates are shown in the figure.

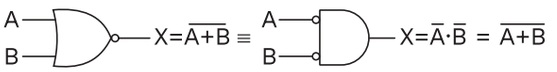


Fig. 2.16 Two Level NOR-NOR Representation

NOR-NOR implementation:

The implementation of a Boolean function with NOR-NOR logic requires that the function should be a simplified POS expression. In the first implementation, all sum terms are implemented by the NOR gate, and in the second level, all ANDed terms are implemented using the equivalent of the NOR gate. The following steps are used:

1. Simplify the logic expression and convert it into POS form.
2. Draw an OR-AND logic circuit for the above-simplified expression.
3. Replace OR gate by NOR gate and AND gate by invert AND gate.
4. Replace invert AND gate by NOR gate.

Example 9:

Implement the given Boolean function using 2-input NOR-NOR gate logic if the inputs are given in normal and complement form.

$$F = \overline{AC + BC + D}$$

Solution:

Following steps are used:

Step 1:

Express the given Boolean function in POS form by using DeMorgan's theorem. Thus,

$$F = (\bar{A} + \bar{C})(\bar{B} + \bar{C})(\bar{D})$$

Step 2:

Implementing the Boolean function with OR-AND logic.

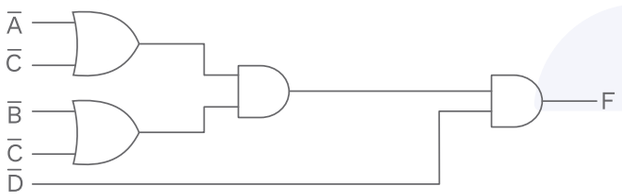


Fig. 2.17

Step 3:

OR gate is replaced by NOR, and AND gate is replaced by bubble AND gate.

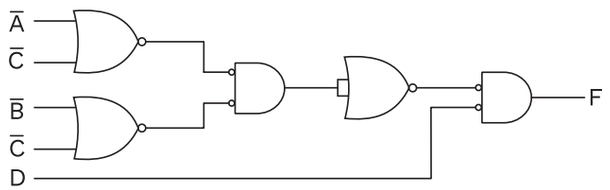


Fig. 2.18

Step 4:

Finally, replace invert AND gate by NOR gate.

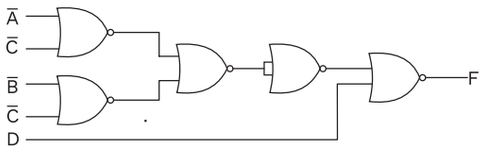


Fig. 2.19

Example 10:

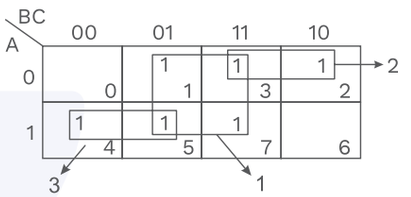
Implement the following Boolean function using 2-input NAND-NAND logic if the inputs are given in normal and complement form.

$$F(A, B, C) = åm(1, 2, 3, 4, 5, 7)$$

Solution:

Step 1:

Simplifying the given standard SOP by using a 3-variable K-map as shown below:



$$F(A,B,C) = C + \bar{A}B + A\bar{B}$$

Fig. 2.20

Step 2:

Implement the logic function with AND-OR logic.

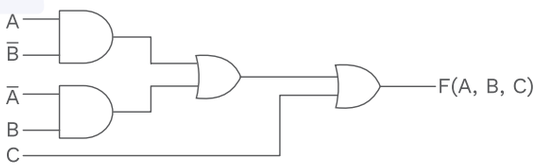


Fig. 2.21

Step 3:

Replace AND gate by NAND gate and OR gate by invert OR gate or bubbled OR gate.

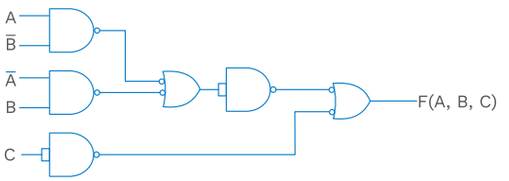


Fig. 2.22

Step 4:
Replace bubble OR gate by NAND gate.

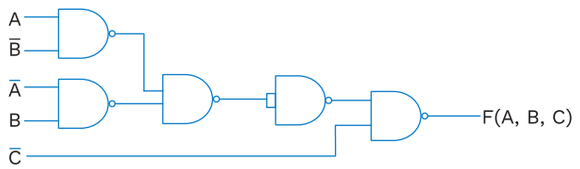


Fig. 2.23

Example 11:

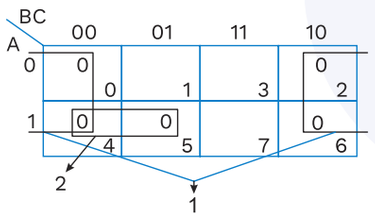
Implement the following Boolean function using 2-input NOR-NOR logic if the inputs are given in normal and complement form.

$$F(A, B, C) = pM(0, 2, 4, 5, 6)$$

Solution:

Step 1:

Simplify by using a suitable K-map as below:



$$F = C(\bar{A} + B)$$

Fig. 2.24

Step 2:
Implement the logic function with OR-AND logic.

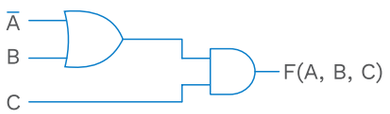


Fig. 2.25

Step 3:
Replace OR gate by NOR gate and AND gate by invert AND gate.

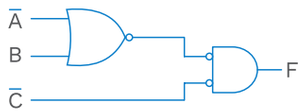


Fig. 2.26

Step 4:
Replace invert AND gate by NOR gate.

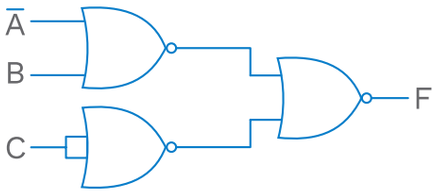


Fig. 2.27

Multilevel 'NAND' and 'NOR' gate Logic

We have studied that the standard form of expression Boolean functions results in two-level implementation. Sometimes, the design of digital systems results in gating structures with three or more levels.

The most common procedure in the design of multilevel circuits is to express the Boolean function in terms of AND, OR and complement operation. These functions can be implemented with AND-OR-NOT gates.

Multilevel NAND implementation:

The following steps are followed to implement multilevel NAND implementation.

1. Draw the logic diagram for the given Boolean expression using AND, OR and NOT gate.
2. Replace all AND gates by NAND gates with bubbled AND (Invert AND) symbols and all OR gates by NAND gates with bubbled OR (Invert OR) graphic symbols.
3. All the bubbles in the diagram have to be carefully checked, i.e., see whether all the bubbles along the same line in the diagram are compensated by another small circle or not. If not, then insert an inverter (one-input NAND gate) or complement the input variable. The invert OR gate is also replaced by a NAND gate.

Example 12:

Perform the NAND implementation of the following function if the inputs are given in normal and complement form.

$$F(A, B, C, D) = A(BC + D) + A\bar{B}$$

Solution:

Step 1:

Implement the logic function with AND, OR and NOT gates.

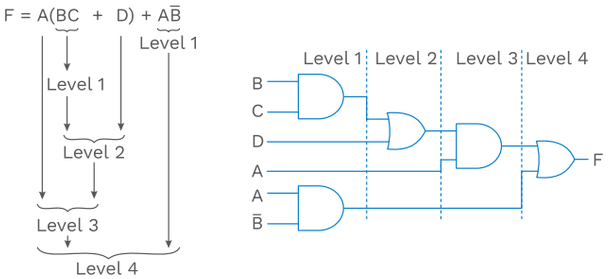


Fig. 2.28

Step 2:

Replace the AND gates by NAND gates and OR gates by invert OR gates.

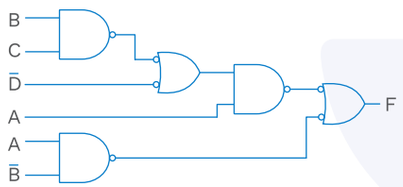


Fig. 2.29

Step 3:

The input D line has only one bubble. Now, the input D is complemented. Replace invert OR by NAND.

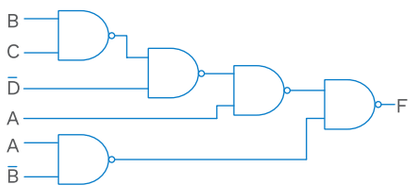


Fig. 2.30

Multilevel NOR implementation:

The following steps are followed in the NOR implementation.

1. Draw the logic diagram for the logic expression using AND, OR and NOT gates.
2. Replace all AND gates by invert AND gates and all OR gates by NOR gates.
3. All the bubbles in the diagram have to be carefully checked. Insert an inverter equivalent NOR gate for every bubble that is not compensated by another bubble along the same line. Finally, replace the invert AND gate by NOR gate.

Example 13:

Perform the NOR implementation of the following Boolean function:

$$F(A,B,C,D) = A(BC + D) + A\overline{B}$$

Solution:

Step 1:

Implement the logic function by AND, OR and NOT gates.

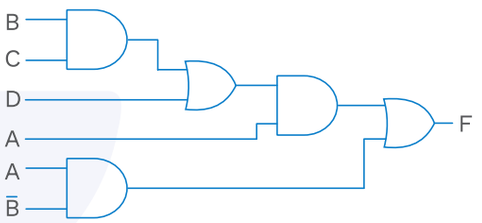


Fig. 2.31

Step 2:

Now, replace OR gate by NOR gate and AND gate by invert AND gate.

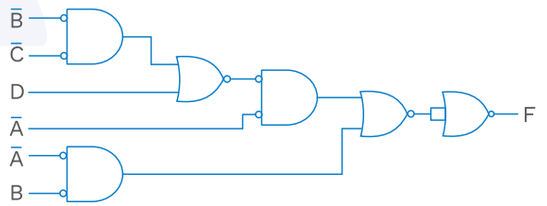


Fig. 2.32

Step 3:

Replace invert AND by NOR.

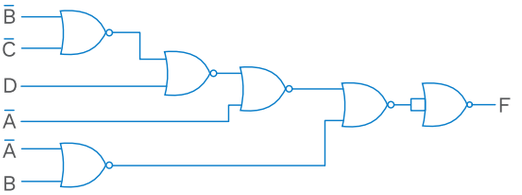


Fig. 2.33

Chapter Summary



- Boolean algebra or logical algebra was invented by George Boole in 1854.
- It is used to analyse and simplify the digital (logic) circuits. It uses only binary numbers, i.e., 0 and 1.
- Variables used in Boolean algebra can only have two values.
- For HIGH, binary 1 is used, and for low, binary 0 is used.
- Boolean functions can be represented by using truth tables.
- Truth tables are basically a means to express the characteristics of Boolean functions as well as logic circuits.
- A Boolean function and its digital representation are properly matched with each other.
- AND, OR and NOT are called as basic logic gates.
- NAND and NOR are called as universal gates.
- An X-OR gate is also called an anti-coincidence gate or inequality detector.
- An X-NOR gate is also called a coincidence gate or equality detector.

Practice Questions

1. The number of Boolean functions that n variables can generate are equal to:

(A) $22n-1$

(B) 2^{2^n}

(C) $2n-1$

(D) $2n$

2. The logical expression $y = A + \bar{A}B$ is equivalent to:

(A) $y = AB$

(B) $\bar{A}B$

(C) $y = \bar{A} + B$

(D) $y = A + B$

3. The minimised form of the logical expression $(\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + \bar{A}BC + AB\bar{C})$ is:

(A) $\bar{A}\bar{C} + B\bar{C} + \bar{A}B$

(B) $A\bar{C} + \bar{B}C + \bar{A}B$

(C) $\bar{A}C + \bar{B}C + \bar{A}B$

(D) $\bar{A}C + B\bar{C} + \bar{A}B$

4. The simplified form of the logical expression $F = \overline{\left(A + \overline{BC}\right)}\left(A\bar{B} + ABC\right)$ is:

(A) 1

(B) $\bar{A}B$

(C) ABC

(D) 0

5. If the functions W, X, Y and Z are as follows:

$W = R + \bar{P}Q + \bar{R}S$

$X = PQ\bar{R}\bar{S} + \bar{P}Q\bar{R}\bar{S} + P\bar{Q}\bar{R}\bar{S}$

$Y = RS + \overline{PR + \bar{P}\bar{Q} + \bar{P}\bar{Q}}$

$Z = R + S + \overline{PQ + \bar{P}\bar{Q}\bar{R} + P\bar{Q}\bar{S}}$

Then,

(A) $W = Z, X = \bar{Z}$

(B) $W = Z, X = Y$

(C) $W = Y$

(D) $W = Y = \bar{Z}$

6. The Boolean expression $AC + B\overline{C}$ is equivalent to:

(A) $\overline{A}C + B\overline{C} + AC$

(B) $\overline{B}C + AC + B\overline{C} + \overline{A}C\overline{B}$

(C) $AC + B\overline{C} + \overline{B}C + ABC$

(D) $ABC + \overline{A}B\overline{C} + AB\overline{C} + A\overline{B}C$

7. The Boolean expression for the truth table shown is:

A	B	C	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

(A) $B(A + C)(\overline{A} + \overline{C})$

(B) $B(A + \overline{C})(\overline{A} + C)$

(C) $\overline{B}(A + \overline{C})(\overline{A} + C)$

(D) $\overline{B}(A + C)(\overline{A} + \overline{C})$

8. The Boolean expression $Y = \overline{A}\overline{B}\overline{C}D + \overline{A}BC\overline{D} + A\overline{B}\overline{C}D + AB\overline{C}\overline{D}$ can be minimised to:

(A) $Y = \overline{A}\overline{B}\overline{C}D + AB\overline{C} + A\overline{C}D$

(B) $Y = \overline{A}\overline{B}\overline{C}D + BC\overline{D} + A\overline{B}\overline{C}D$

(C) $Y = \overline{A}BC\overline{D} + \overline{B}\overline{C}D + A\overline{B}\overline{C}D$

(D) $Y = \overline{A}BC\overline{D} + \overline{B}\overline{C}D + AB\overline{C}\overline{D}$

Practice Solutions

1. Solution: (B) **2. Solution: (D)** **3. Solution: (A)** **4. Solution: (D)**

5. Solution: (A) **6. Solution: (D)** **7. Solution: (A)** **8. Solution: (D)**